CS3523 Operating Systems Programming Assignment 3: Virtual Memory and Page Replacement

Your Name Roll Number

1 Introduction

This assignment extends the xv6 operating system by implementing a demand-paged virtual memory system with swapping and page replacement. The system is designed to operate under constrained physical memory ($\mathrm{MAX}_{F}RAMES = 32)andsupportslazyallocation, swap - in, swap - out, andevictionpolicies$.

The implementation integrates concepts from both memory management and scheduling, including a Clock-based replacement algorithm and scheduler-aware eviction behavior.

2 Design Overview

2.1 Lazy Allocation

Memory allocation is performed lazily using a custom system call `sbrklazy()`. Pages are not physically allocated until they are accessed, which triggers a page fault handled in `vmfault()`.

2.2 Frame Table

A global frame table is maintained to track physical frames:

```
struct frame_entry {
int used;
struct proc *proc;
uint64 va;
uint64 pa;
int ref_bit;
};
```

Each entry stores:

- Owner process
- Virtual address
- Physical address
- Reference bit (Clock algorithm)

2.3 Swap System

Swap is implemented using an in-memory array of slots:

```
struct swap_slot {
int used;
struct proc *proc;
uint64 va;
char data[PGSIZE];
};
```

3 Page Fault Handling

Page faults are handled in `usertrap()`:

```
if(vmfault(p->pagetable, va, write) < 0)
setkilled(p);
```

3.1 vmfault() Cases

1. Lazy Allocation: allocate and map new page
2. Swap-in: load page from swap
3. Reference update: set accessed bit

4 Page Replacement Policy

A Clock (Second-Chance) algorithm is used:

- Circular scan over frame table
- If $\text{ref}_bit = 1ßresetto0$
- If $\text{ref}_bit = 0ßevict$

5 System Call: getvmstats

```
struct vmstats {
int page_faults;
int pages_evicted;
int pages_swapped_in;
int pages_swapped_out;
int resident_pages;
};
```

This system call is used extensively for testing and verification.

6 Experimental Evaluation

The implementation was validated using extensive test cases including both provided and custom tests.

6.1 Basic Virtual Memory Test

From usertests (vmbasic) :

- 50 pages allocated (greater than 32 frames)
- Page faults increased
- Evictions occurred
- Swap-out and swap-in verified

Result:

```
test vmbasic: OK
ALL TESTS PASSED
```

6.2 Fork Consistency Test

- Parent initializes memory
- Two children modify independently
- Parent data remains unchanged

Result:

```
test vmfork2: OK
ALL TESTS PASSED
```

6.3 Swapping Behavior

```
Resident Pages: 32
Faults: 40
Swapped Out: 8
Swapped In: 1
```

Observation:

- Resident pages capped at 32
- Swap-out triggered correctly
- Swap-in occurs on re-access

6.4 Thrashing Test

```
Faults: 104
Swapped Out: 72
Swapped In: 71
```

Observation:

- High fault rate indicates thrashing
- Clock algorithm cycles correctly

6.5 Memory Shrink and Reuse

```
SUCCESS: Memory successfully freed and reused
```

Observation:

- uvmunmap correctly frees frames
- No memory leaks detected

6.6 Exec Cleanup Test

```
Exec cleanup test finished.
```

Observation:

- Swap slots cleaned after exec

6.7 Priority-Aware Eviction

```
High Priority Swapped Out: 0
```

Observation:

- High priority processes protected
- Scheduler-aware eviction works

6.8 Custom VM Tests

6.8.1 Page Fault Only

```
faults=10 evicted=0 resident=10
```

6.8.2 Eviction

```
faults=50 evicted=18 swapped_out=18 resident=32
```

6.8.3 Swap-In Integrity

```
data integrity: PASS
swapped_in=48
```

6.8.4 Priority Comparison

```
child: evicted=28
parent: evicted=0
```

7 Key Observations

- Lazy allocation reduces unnecessary memory usage
- Clock algorithm balances performance and simplicity
- Swap system ensures correctness under memory pressure
- Scheduler-aware eviction improves fairness

8 Challenges Faced

- Kernel panics due to incorrect PTE handling
- Maintaining consistency between frame table and page tables
- Debugging swap-in/out edge cases
- Handling fork with swapped pages

9 Limitations

- Swap is memory-backed, not disk-based
- No copy-on-write optimization
- Clock algorithm is not optimal under heavy thrashing

10 Conclusion

The implementation successfully integrates lazy allocation, page replacement, and swapping into xv6. Extensive testing confirms correctness, robustness, and adherence to expected behavior under various workloads.

11 References

- xv6 Source Code
- MIT xv6 Documentation
- GEN AI usage for code understanding and implementation parts and report and test case generation